CT-200 Traceability API: Approach

Repository: https://github.com/thenameispravardhan/ct200-traceability

0. At a glance

Problem. A medical-device manual changes between revisions. QA test cases written against the old text quietly stop being true ("auto-deflates within 2 seconds" when v2 says 1.5). The system has to notice this, per test case, and say why.

```
POST /documents/{doc}/versions   (raw PDF)
  -> pdf_tree.parse ............ section nodes + irregularity warnings
  -> versioning.ingest ......... SQLite: version, immutable node rows,
                                 logical_id matched to the previous version,
                                 diff + invalidated facts per changed section
POST /selections ............... version-pinned set of node rows
POST /selections/{id}/generations
  -> llm.generate .............. JSON -> schema, citation, grounding checks
                                 retry x3; still invalid -> 502, nothing stored
  -> generations/<id>.json ..... exact source text + hashes copied in
GET  /selections/{id}/generations   (or /nodes/{id}/generations)
  -> versioning.staleness ...... against the latest version, per test case:
                                 current | review | stale (+ invalidated facts)
```

Result on the real v1 → v2 manuals:

Measure	Result
Sections parsed	27 in v1, 28 in v2; every irregularity flagged, none silently repaired
v2 re-ingestion	23 unchanged, 4 modified (2.1.1.1, 3.2, 4.2, 4.3), 1 added (5.3), 0 removed; no false changes from page breaks
Facts each change invalidates	2.1.1.1: `300`, `15%` · 3.2: `40 mmHg` · 4.2: `2 seconds` · 4.3: `E5`
Test cases from table 4.2	the E3 test (asserts "2 seconds") → **stale**; E1, E2, E4, E5 tests → **review**
Tests	12 (6 parser irregularity tests, 6 end-to-end API tests including the LLM failure modes)

Three ideas that carry the design:

1. **Numbering is identity, styling is evidence.** Headings are found by bold + section number, never font size. Parents come from the number prefix, so out-of-order and level-skipping sections still land in the right place, with a warning.
2. **An LLM answer is untrusted input.** It must parse, match the schema, cite only sections it was given, and only assert numbers those sections contain. If any check fails, the model is told exactly why and retried. After 3 failures nothing is stored.
3. **Staleness is a property of a claim, not of a section.** Each changed sentence is paired with its rewrite to find exactly which facts were dropped. A test goes stale only if it asserts one of those facts. Nothing is ever auto-cleared: any change to a cited section means at least `review`.

Run it (with no API key it uses a labelled offline stub; set `LLM_API_KEY` for a real model):

```
pip install -r requirements.txt
pytest -q                  # 12 tests
uvicorn main:app           # then, in a second terminal:
```

```
python demo.py            # v1 -> selection -> generation -> v2 -> staleness
```

1. Document parsing

Why the text layer and not OCR

Before writing any parser I dumped every text line of both PDFs with its font, size and position, and rendered each page to an image to compare. Both manuals are born-digital: every page has a text layer, there are no images, and the text layer matches the rendered page exactly, including ±, ≥, – and the table cells. OCR would only add recognition errors (≥→>, ±3→+3, a dropped en dash in `0–299 mmHg`), and those are the threshold values the test cases depend on. It would also lose the bold/regular distinction that the heading detection relies on.

So `pdf_tree.py` reads the text layer with **PyMuPDF**, including its `find_tables()` for tables. OCR is not silently skipped, though: a page without a text layer makes ingestion fail with `422 page N has no text layer; scanned pages are not supported`, and embedded images raise a document-level warning. With more time, the fallback would be PyMuPDF's Tesseract hook (`page.get_textpage_ocr`) for image-only pages. Without bold information, those pages would rely on numbering and font size alone.

This departs from the brief's "OCR-based" wording. I think it is the right call for this document, and the refusal path keeps the unsupported case explicit.

Hierarchy reconstruction

- **A heading is a line set entirely in bold that starts with a section number** (`^\d+(\.\d+)*\.?\s`). Font size is deliberately not used, because two headings (`3.2`, `2.1.1.1`) are set in 11 pt, the same size as body text.
- **The parent is the closest existing numeric prefix**, not "the previous heading one level up". The numbering is authoritative even when the visual order is not, so `3.3` belongs under `3` even though it appears after `3.4`, and `2.1.1.1` lands under `2.1` because `2.1.1` does not exist.
- Every irregularity is **attached to the node as a warning** and exposed in the API (`warnings` on every node), instead of being silently repaired.
- **Body text** is grouped into blocks in reading order:
  - paragraphs (joined while the vertical gap is under 4 pt);
  - lists (a line starting `1.`, • and so on, with indented continuation lines folded into the item);
  - tables.

  Each node stores the blocks as JSON and a plain-text `body` rendering.
- **The content hash** is SHA-256 over heading + body with all whitespace collapsed. The section number is excluded, so renumbering is not a content change. Paragraph breaks are excluded too (see 3.1 below).
- Text is NFKC-normalised, which folds the fi/ff ligatures and turns the non-breaking hyphen U+2011 in "CT-200" into –.

Irregularities found in the manual, and how each is handled

Where	What	Handling	Test

`2.1.1.1 Battery Life`	Skips a level: no `2.1.1`. Also set in 11 pt bold, body size	Parent = `2.1`, `level` = 4, warning `skipped level`	`test_skipped_level_...`
`3.4` before `3.3`	Out-of-order numbering	Both children of `3`, document order kept, warning `out of order` on 3.3	`test_out_of_order_...`
`4.2` and `7.1`	Duplicate heading "Error Codes"	Two nodes, distinct ids and hashes, parents 4 and 7, warning on 7.1	`test_duplicate_heading_...`
`3.2 Cuff Inflation`	Heading at body font size	Bold + numbering rule, so it is still a heading	`test_body_sized_bold_heading_...`
`3.3` list `1. Normal: ...`	Numbered list that matches the heading regex	Not bold, so treated as a list. The wrapped 5th item is joined	same test
`2.1`, `4.2`	Tables, with multi-line cells in 4.2	`find_tables` → rows; cell line breaks joined	`test_tables_...`
Every page	`find_tables` also reports each cell's text as its own one-cell "table"	Keep only tables not contained in another table	same test
`3.1` (v1)	Body split across a page break after a full sentence	Two paragraphs in v1, one in v2. The hash ignores whitespace, so 3.1 is **unchanged**	`test_page_breaks_...`
`4.3` (v2), `8.1` (v2)	Sentence continues on the next page	Joined when the previous line did not end a sentence	same test
Level-1 headings	Often the last line of a page, with the body on the next page	The section stays open across pages	implicit in all tests
Title	Three 22 pt lines before section 1	Joined into the version title. Any other pre-heading text becomes a warning	same test
Body	`CT-200` spans split at U +2011; ﬁ/ﬀ ligatures	NFKC + hyphen fold	same test

Content issues the parser cannot and should not "fix", but a QA reviewer should know about:

- 6.2 says *"Anthropic recommends professional recalibration"*. The manufacturer is CardioTrack.
- 4.1's second condition ("exceeds 300 mmHg for longer than 3 seconds") is already covered by the first ("exceeds 299 mmHg at any point"), so the spec is ambiguous about which behaviour wins.
- 3.3 cross-references 2.1 and 4.3. Those references are not modelled as dependencies (see staleness limits).

Process: what went wrong first, and how I found it

- **Font-size heading detection** was my first idea from looking at the opening pages (22/16.5/12.9 pt).

The span dump showed `3.2` and `2.1.1.1` at 11 pt, so I switched to bold + numbering before writing the code.

- **`find_tables()` output had duplicates.** When I printed its raw output for page 2 and 4, each table appeared once properly and then again as many one-cell fragments (`['Pressure', 'range']`, ...). I fixed this with the containment filter.
- **3.1 would have been reported as modified in v2.** Diffing the printed v1 and v2 trees showed v1 renders 3.1 as two paragraphs (page break) and v2 as one. A hash over the raw body would have flagged a false change and made every test case citing 3.1 "stale". I made the hash whitespace-insensitive and added `test_page_breaks_...` to keep it that way.
- **Section numbers inside the compared text.** The first version of the cross-version diff compared `"3.3 Heading\nbody"`. A renumbered section would then have had its number counted as a changed "fact" (a material change). I removed the number from the compared text.
- **Silent swallowing of an unstyled heading.** While writing the decision log, I realised a numbered heading that is *not* bold would merge into the previous section's body with no signal. I added a warning for non-bold numbered lines at the heading margin. Numbered list items are indented, so they do not trigger it.

Validation was layered:

1. the span dump and page renders, compared by eye;
2. a printed indented tree per version (`python pdf_tree.py data/ct200_manual.pdf`) and a `diff` of the v1 and v2 trees, which shows exactly the real edits and nothing else;
3. assertions per irregularity in `tests/test_parser.py`.

2. Data model

SQLite via SQLAlchemy (`store.py`):

- **versions**: `(document, number)` unique, file SHA-256, title, document-level warnings.
- **nodes**: one row per section *per version*. Rows are never updated after ingestion.
  - Tree and text: `parent_id` (same version), `position`, `number`, `heading`, `level`, `page`, `body`, `blocks` (JSON), `content_hash`, `warnings`.
  - Versioning:
    - `logical_id`: the same section across versions;
    - `change`: added / unchanged / modified vs the previous version;
    - `matched_by`: which matching rule linked it.
- **selections** + **selection_items**: a selection references node *rows*. A row belongs to exactly one version and is immutable, so a selection is version-pinned by construction and always resolves to the text it was created against. No snapshot copy is needed.

LLM output goes to a JSON document store (`generations/<id>.json`, written atomically with `os.replace`). Each generation is self-contained:

- links to the selection;
- a **`sources`** array with the exact `node_id`, `logical_id`, `content_hash` and full text it was generated from;

- the validated test cases, each citing `source_node_ids`;
- model, prompt version, attempts, and the raw reply.

Because the source text is copied into the generation, it survives re-versioning and could be audited even if the SQL rows were gone.

Why a JSON store and not MongoDB. Generations are written once, never updated, read by selection or by node, and never joined with the tree. A directory of JSON files gives that with no server to run. The ceiling is the full scan on every read (marked with a `LIMIT:` comment in `store.py`). Past a few thousand generations, or with more than one API process, this belongs in MongoDB with indexes on `selection_id` and `sources.logical_id`. The document shape would not change.

3. Version matching

When version *n+1* is ingested, each new node is matched to an unmatched node of version *n*, in three passes:

1. **Same section number and same heading.** This is the normal case, and it is what keeps the two "Error Codes" sections apart.
2. **Same heading under the same (already matched) parent.** This catches a section that was renumbered.
3. **Identical content hash.** This catches a section that was renamed or moved but whose text is untouched.

A matched node inherits the `logical_id`. Equal hash → `unchanged`; different hash → `modified`, with a diff summary. Unmatched new nodes are `added`; unmatched old nodes are reported as `removed` in the ingest response.

Why this order. For a controlled document, the section number plus title is the identity that engineers and auditors use, so it should win. Fuzzy title matching was rejected: with a duplicate title already in the document, "close enough" matching is how traceability silently attaches to the wrong section.

On v1 → v2 this gives: 23 unchanged, 4 modified (2.1.1.1, 3.2, 4.2, 4.3), 1 added (5.3), 0 removed.

Where it breaks:

- **Renumber + retitle + edit in one revision.** None of the rules fires, so the section is seen as removed + added. Traceability is lost, but in the safe direction: generations citing it show `removed` → `stale`.
- **A section split in two.** One half matches by number + heading, the other is "added". Test cases written against the old whole are compared with only one half.
- **Two sections swap titles while keeping their numbers.** Neither matches by rule 1, rule 2 matches them by heading, so each logical id follows the *title*. That is probably right, but it is a judgement call.
- **The same number + heading twice in one version.** Matching is first-come; the parser warns `duplicate section number`.
- Matching only compares against the immediately previous version. A section deleted in v2 and restored in v3 gets a new logical id.

4. Change classification and staleness

`change_summary(old, new)` returns:

- `severity`: **material** if any *fact token* was added or removed, otherwise **wording**. Fact tokens are numbers with units (`2 seconds`, `15%`, `40 mmHg`), codes (`E6`) and requirement words (`not`, `cannot`, `must`, `only`, `except`, `within`, `before`, ...);
- the facts removed and added;
- **`invalidated_facts`**: facts that an old sentence stated and its rewrite no longer states. Each rewritten old sentence is paired with its most similar new sentence (word overlap ≥ 0.5) and their facts are compared. A sentence with no partner counts as deleted, so all its facts are invalidated;
- word-level similarity;
- the changed sentences as `-`/`+` lines.

Staleness is **computed at retrieval time**, not stored, so it is always against the latest version. For each source node of a generation, look up the same `logical_id` in the latest version:

- equal hash → `current`;
- different hash → `changed` (+ diff summary);
- missing → `removed`.

Each test case is judged on what it claims, not on which section it came from. A test's claims are the fact tokens in its own title, steps and expected result. Its status is:

- **stale**: a cited section was removed, or one of the test's own facts is among the cited sections' `invalidated_facts`. The facts in question are returned on the test case.
- **review**: a cited section changed, but nothing this test asserts was invalidated.
- **current**: every cited section is byte-for-byte unchanged (equal hash).

The generation as a whole gets the worst status of its test cases. All of this is shown on every retrieval endpoint, per source and per test case.

Concretely, on v1 → v2 (shown by `demo.py` and asserted in `tests/test_api.py`):

Test case (from v1)	v2 change	Status
E3 auto-deflates "within 2 seconds" (4.2)	E3 row now says 1.5 s	**stale**, invalidated `2 seconds`
E1 aborts and displays E1 (4.2)	same table, other rows changed, E6 added	**review**
Inflates "in 40 mmHg increments" (3.2)	increment is now 30; "40 mmHg" survives only in a new explanatory sentence	**stale**, invalidated `40 mmhg`
Aborts above a maximum of 299 mmHg (3.2)	same rewritten sentence, but 299 survived the rewrite	**review**
Deflation steps ≈ 3 mmHg (3.2)	that sentence untouched	**review**
Profile selection (3.1)	no change (only the page break differs)	**current**

Real response excerpt (`GET /generations/{id}` after v2 is ingested; generated from a selection of 4.2 only, trimmed):

```
"staleness": {"status": "stale", "generated_from_version": 1, "checked_against_version": 2,
  "sources": [{"number": "4.2", "status": "changed", "severity": "material",
               "facts_removed": ["2 seconds"], "facts_added": ["1.5 seconds", "e6", "e6", "not"],
               "invalidated_facts": ["2 seconds"]}]},
"test_cases": [
  {"id": "TC-1", "expected_result": "E1 | ... | Aborts measurement, displays E1",
   "staleness": "review", "invalidated_facts": []},
  {"id": "TC-3", "expected_result": "E3 | ... | Auto-deflates within 2 seconds, displays E3",
   "staleness": "stale", "invalidated_facts": ["2 seconds"]}, ...]
```

How that rule evolved (each step was forced by a real case in this manual):

1. **Section-level** (first version): any material change to a cited section made every test citing it stale. Table 4.2 broke this. One row changed, so all five error-code tests went stale, including the four whose rows were untouched.
2. **Vanished tokens**: stale only if a fact the test asserts disappeared from the section. 3.2 broke this. "40 mmHg" is still in v2 (in the new sentence "reduced from the original 40 mmHg"), so the 40 mmHg increment test looked unaffected, which is a false negative.
3. **Facts in rewritten sentences**: stale if the asserted fact sits in any rewritten sentence. This catches 3.2, but over-flags: 299 mmHg is in the same sentence and did not change.
4. **Paired sentences** (current): pair each rewritten sentence with its rewrite and invalidate only what the rewrite dropped. 3.2 gives exactly `40 mmHg`; the 299 test stays at review.

Nothing is auto-cleared. Any change to a cited section yields at least `review`. Only an identical hash yields `current`.

Does a one-word change count the same as a changed pressure threshold? No, but it is also never silently cleared. The heuristic can tell "2 seconds → 1.5 seconds" from "approximately → about". It cannot reliably tell that "increases" → "decreases" or "systolic" → "diastolic" changes the requirement. So wording-only changes become `review` (a human must look), never `current`. Only an identical hash yields `current`.

Known limits:

- **Facts are tokens, not meaning.** "increases" → "decreases" or "systolic" → "diastolic" carry no fact token. They still yield `review` (the section changed), never `stale`, so a human has to catch them.
- **Sentence pairing is a threshold.** A rewrite that changes more than half of a sentence's words is treated as a deletion, so all of its facts are invalidated (over-flagging, the safe side). A sentence split into two can pair with only one half.
- **Unit spelling.** "2 seconds" and "2 second" match, but "2 s" does not. A test written as "2 s" would drop from stale to review (never to current).
- **The fact vocabulary is tuned to this manual** (mmHg, bpm, %, seconds, E-codes, requirement words). Another device's units would need adding.
- **Dependencies outside the selection are invisible.** 3.3 says "see 2.1, 4.3". A change in 4.3 does not stale a test generated only from 3.3.
- **Additions elsewhere.** New section 5.3 (Data Export) cannot stale anything, though it may deserve new tests. That is a coverage question, not a staleness one.

5. LLM generation

Prompt (`llm.py`, version `tc-v1`):

- The system prompt casts the model as a QA engineer for a regulated device. It must use *only* the given text, never invent values, and write executable steps and measurable expected results that quote the exact values. It should prefer boundary and failure-mode tests.
- The output is a single JSON object with a stated shape.
- The input labels each section `[node N] Section 4.2 Error Codes ...`, so the model can cite its sources by id.

Structured output and failure policy:

- The request uses `response_format: json_object` where the provider supports it, but nothing trusts it. Markdown fences are stripped, then the reply is validated with Pydantic:
  - 3–5 test cases;
  - non-empty title, 1–12 steps, non-empty expected result, at least one source id;
  - every `source_node_id` must be one of the selection's nodes. A citation of a node the model was not given is a hallucination and is rejected;
  - **grounding**: every number in an `expected_result` must appear in the sections that test cites. "Auto-deflates within 5 seconds" against a spec that says 2 seconds is rejected and retried, with the offending value named in the feedback.

    For a medical device, a plausible invented threshold is the most dangerous kind of LLM output: it is valid JSON, reads well and would pass a schema check. This turns it into a hard error. The same numbers are what staleness keys on later, so the check also guarantees that every asserted value can be traced back to the text. - On a parse or validation failure, the model is shown its own reply plus the exact validation error and asked again, up to **3 attempts**. Transport errors (429/5xx/timeouts) back off 2 s, then 4 s, and count as attempts.
- If all attempts fail: **HTTP 502** with every attempt's error and the last raw reply, and **nothing is stored**.
- Partial salvage (keeping the 2 valid cases out of 4) was rejected on purpose. A suite stored as "generated" must have passed the whole contract; otherwise retrieval would show half-valid output as if it were real.

Same selection submitted twice: the request is fingerprinted as SHA-256 of (prompt version, model, source node ids + hashes + text). If a successful generation with that fingerprint exists, it is returned with `reused: true`, and the LLM is not called.

- **Why reuse.** Selections are immutable, so the inputs are identical. A second call would only produce a different random answer to the same question, which pays for and creates two competing "official" suites for one requirement set.
- **Getting a new one anyway.** `?force=true` creates an additional generation; old ones are never deleted.
- **What breaks reuse.** Changing the prompt version or the model changes the fingerprint, so those generate fresh.

Selections follow the same rule: the same name + the same resolved node set returns the existing selection.

Provider: any OpenAI-compatible chat API, configured with `LLM_BASE_URL` / `LLM_MODEL` / `LLM_API_KEY` (Groq by default). Without a key, an **offline stub** turns sentences containing numbers into test cases. It exists so the pipeline (validation, storage, staleness) runs end to end offline and in tests. It is labelled `model: offline-stub` in every generation and is not presented as LLM output. **The real-provider path has not**

been exercised against a live API in this repo (no key was available while building it). The validation and retry logic around it is tested with injected malformed replies (`tests/test_api.py`).

6. Stack

FastAPI, Pydantic, SQLAlchemy + SQLite, as asked. The one deviation is the JSON-file store instead of MongoDB; the reasons are in section 2.

7. What I'd do with more time

- Run the generation step against several real models and record how often each validation rule fires. Today the retry and grounding logic is proven with injected bad replies, not with measured model behaviour.
- Model explicit cross-references ("see 2.1, 4.3", "Section 4.2") as dependency edges, and include them in staleness.
- An OCR fallback for scanned pages, and a two-column layout check.
- A golden-tree snapshot (`expected_tree_v1.json`) compared in CI, so any parser change that moves a node shows up as a diff.
- Extend grounding beyond numbers. Today an expected result can't invent a value, but it can still invent a *behaviour* ("displays a warning"). An entailment check (rules or a second model) against the cited sentence would close that.
- MongoDB for generations, and a lock or queue around ingestion.

8. Decision log

1. What is most likely to give wrong results silently, and how would I catch it?

The heading detection in the parser. Everything downstream (hashes, matching, selections, staleness) trusts that each node's text really belongs to that section. If a future revision sets a heading in regular weight, or bolds a numbered sentence, the text is attributed to the wrong node and nothing errors. The tree looks plausible, the diff says "section X modified, section Y removed", and test cases get generated from the wrong text.

Catching it today: - the parser warns on non-bold numbered lines at the heading margin, and on skipped, duplicate and out-of-order numbers;

- the ingest response shows every added and removed section, so an unexpected "removed" stands out.

What I would add: a numbering-continuity check (3.1, 3.2, 3.4 → "3.3 missing"), and the golden-tree snapshot test.

2. Where did I choose simplicity over correctness because of time, and what breaks first in production?

The meaning of a change is approximated with a regex vocabulary of "fact tokens" (numbers + units, error codes, requirement words) instead of real semantic comparison. It is transparent, testable, deterministic and free. But it only knows the units of this manual, and it cannot see a meaning flip that involves no such token.

Those cases land in `review`, which is honest but relies on a human reading the diff.

What breaks first in production is reviewer trust in `review`: if most `review` flags turn out to be harmless rewording, people stop reading them, and then the one semantic flip gets missed. I would measure the false-review rate on real revisions before relying on it.

Operationally, the next things to break are: - the JSON store's full scan per request; - concurrent ingests of the same document. The unique `(document, number)` constraint turns the race into an error rather than corruption, but it is still an error.

3. One input I did not handle, and what the system does with it.

A multi-column page. The parser sorts lines by vertical position (it assumes a single column, and says so in a `LIMIT:` comment), so a two-column page would interleave the lines of both columns into nonsense paragraphs.

This one is **not** detected: it would produce wrong body text and hashes with no warning. Only the heading warnings might hint at it, if headings end up in odd places.

By contrast, a scanned PDF (no text layer) is detected and refused with a 422 naming the page.